

React Hooks

React Hooks are tools that allow you to use state and other React features without writing class components. They're designed to simplify your code and make it easier to share logic across components. **React Hooks are tools that let your component “remember things” and “do things”.**

- You can only use them inside function components.
- You have to use them at the top of your component, not inside loops or conditions.

useState

Remember something

The useState hook allows you to add state to a functional component. It takes an initial value as an argument and returns an array with two elements: the current state value and a function to update it.

```
const [count, setCount] = useState(0);
```

Hey React, remember this number, and update it when I ask.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(count + 1);
  }
  const decrement = () => {
    setCount(count - 1);
  }

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>decrement</button>
    </div>
  );
}

export default Counter
```

useEffect

The useEffect hook allows you to perform side effects in a functional component. Side effects include things like fetching data from an API, updating the DOM, or subscribing to an event.

Runs code when:

- Component loads
- Data changes
- Component is removed

```
function DataFetcher() {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('http://www.omdbapi.com/?i=tt3896198&apikey=22e614d2')
      .then(response => response.json())
      .then(result => setData(result));
  }, []);

  if (!data) return <p>Loading...</p>;

  return (
    <div>
      <h2>{data.Title}</h2>
      <p>Year: {data.Year}</p>
      <p>Director: {data.Director}</p>
    </div>
  );
}
```

useContext

[Props for local data, context for global data.](#)

The useContext hook allows you to access a context object in a functional component. Context is a way to pass data down the component tree without having to pass props manually.

Context object: A context object is like a **shared storage box** that React keeps at the top level so any component **can read from it, no matter how deep it is**.

props	useContext
-------	------------

<pre> App ↓ Navbar ↓ Menu ↓ MenuItem </pre>	<pre> <UserContext.Provider value={user}> <App /> </UserContext.Provider> </pre>
Even if only MenuItem needs the data, you must pass props through every level .	const user = useContext(UserContext);
Scope: Parent → child	Scope: Global

Example idea: Instead of passing it through 10 components, you use context

- logged-in user
- theme (dark/light)
- language
- auth token

```
const ThemeContext = React.createContext('dark');
```

```
function ThemeButton() {
  const theme = useContext(ThemeContext);

  return (
    <button
      style={{
        background: theme === 'dark' ? 'black' : 'white',
        color: theme === 'dark' ? 'white' : 'black'
      }}
    >
      Toggle Theme
    </button>
  );
}
```

If you want to keep it shared across multiple components, you must use **ThemeContext.Provider** somewhere at the top. [self study]

useRef

The useRef hook allows you to create a mutable ref object that persists for the lifetime of the component. You can use it to store and access values that don't trigger a re-render.

- Access DOM elements directly.

```
function TextInput() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus(); // direct DOM access
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Type something" />
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

- Persist values across renders without causing re-renders.

Calling `const reference = useRef(initialValue)` with the initial value returns a special object named `reference`. The reference object has a property **current**: you can use this property to read the reference value `reference.current`, or update `reference.current = newValue`. Between the component re-renderings, the value of the reference is persistent. [Updating a reference, contrary to updating state, doesn't trigger component re-rendering](#)

```
import { useRef, useState } from 'react';
```

```
function RefPersistenceDemo() {
  const [dummy, setDummy] = useState(0); // Just to trigger re-renders
  const countRef = useRef(0);
  const renderCount = useRef(0);

  // Track re-renders
  renderCount.current += 1;

  const incrementRef = () => {
    countRef.current += 1; // Updates persist but won't trigger re-render
    console.log(`countRef after increment: ${countRef.current}`);
  };

  const forceReRender = () => {
    setDummy(prev => prev + 1); // Triggers re-render
  };
}
```

```

return (
  <div>
    <h3>useRef Persistence Demo</h3>
    <p>countRef.current: {countRef.current}</p>
    <p>Render count: {renderCount.current}</p>
    <p>Dummy state (triggers re-render): {dummy}</p>

    <button onClick={incrementRef}>Increment Ref (no re-render)</button>
    <button onClick={forceReRender}>Force Re-render</button>
  </div>
);
}
export default RefPersistenceDemo;

```

Examples

Timer IDs

WebSocket instance

External library object

Previous value

Storing a timer ID: We use useRef to store the interval ID because it must persist across renders but does not affect the UI, so it should not trigger re-rendering

useCallback

The useCallback hook allows you to memoize a function so that it's only re-created when its dependencies change. This can help improve **performance** by preventing unnecessary re-renders.

Without useCallback():

```
import { useState } from "react";
import Todos from "../Todos";

const UseCallbackExample = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);

  const increment = () => {
    setCount((c) => c + 1);
  };

  const addTodo = () => {
    setTodos((t) => [...t, "New Todo"]);
  };
  return (
    <>
      <Todos todos={todos} addTodo={addTodo} />
      <hr />
      <div>
        Count: {count}
        <button onClick={increment}>+</button>
      </div>
    </>
  );
};

export default UseCallbackExample;

import { memo } from "react";
const Todos = ({ todos, addTodo }) => {
  console.log("child render");
  return (
    <>
      <h2>My Todos</h2>
      {todos.map((todo, index) => {
        return <p key={index}>{todo}</p>;
      })}
      <button onClick={addTodo}>Add Todo</button>
    </>
  );
};

export default memo(Todos);
```

In React, `memo` is a higher-order component (HOC) used for performance optimization by memoizing a functional component, preventing it from re-rendering unless its props change.

The Problem Without `useCallback`

1. When `count` changes, the `App` component re-renders.
2. On each re-render, the `addTodo` function is **recreated** (a new function reference is generated).
3. The `Todos` component is wrapped with `memo()`, which means it will only re-render if its props change.
4. However, since `addTodo` is recreated on every `App` render, it becomes a **new prop** to `Todos`, causing `Todos` to re-render even when `todos` hasn't changed.
5. This defeats the purpose of `memo()`.

How `useCallback` Helps

By wrapping `addTodo` with `useCallback`, you **memoize the function**, ensuring it keeps the same reference across renders unless its dependencies change.

Without `useCallback` :

- Clicking `+` (increment count) → `App` re-renders → `addTodo` is recreated → `Todos` re-renders unnecessarily.

With `useCallback` :

- Clicking `+` → `App` re-renders → `addTodo` reference stays the same → `Todos` does **not** re-render.

With useCallback():

```
import { useState } from "react";
import Todos from "../Todos";

const UseCallbackExample = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState([]);

  const increment = () => {
    setCount((c) => c + 1);
  };

  ⚡ const addTodo = useCallback(() => {
    setTodos((t) => [...t, "New Todo"]);
  }, []);

  return (
    <>
      <Todos todos={todos} addTodo={addTodo} />
      <hr />
      <div>
        Count: {count}
        <button onClick={increment}>+</button>
      </div>
    </>
  );
};

export default UseCallbackExample;
```

Custom hooks

Custom Hooks in React are JavaScript functions that allow you to extract and reuse stateful logic across different functional components, leading to cleaner, more modular, and maintainable code. They are a convention rather than a built-in feature, and their primary purpose is to share logic without altering the component hierarchy.

Naming Convention: A custom hook's name must always start with the prefix `use` (e.g., `useFetch`, `useLocalStorage`). This convention allows React's linter to enforce the rules of hooks.

Use Other Hooks: Custom hooks can call one or more of the built-in React hooks like `useState`, `useEffect`, `useContext`, etc., to manage state, handle side effects, or access context.

Rules of Hooks Apply: The general Rules of Hooks apply to custom hooks as well:

Hooks must only be called at the top level of a function component or another custom hook, not inside loops, conditions, or nested functions.

Return Values: Unlike components, custom hooks do not return JSX. They can return anything that is useful to the component using them, such as state variables, functions, or objects.

Example:

[useCounter.js](#)

```
import { useState } from 'react';

function useCounter(initialValue = 0) {
  const [count, setCount] = useState(initialValue);

  const increment = () => setCount(prevCount => prevCount + 1);
  const decrement = () => setCount(prevCount => prevCount - 1);
  const reset = () => setCount(initialValue);

  return { count, increment, decrement, reset }; // Return as an object or array
}

export default useCounter;
```

[CounterComponent.js](#)

```
import React from 'react';
import useCounter from './useCounter';

function CounterComponent() {
  const { count, increment, decrement, reset } = useCounter(10); // Initial value of 10

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
      <button onClick={reset}>Reset</button>
    </div>
  );
}

export default CounterComponent;
```

Conditional Rendering

Conditional rendering in React is the practice of displaying different UI elements or components based on specific conditions.

```
function UserGreeting(props) {  
  return <h1>Welcome back!</h1>;  
}
```

```
function GuestGreeting(props) {  
  return <h1>Please sign up.</h1>;  
}
```

```
export { UserGreeting, GuestGreeting };
```

```
import { UserGreeting, GuestGreeting } from './GreetingComponents';
```

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  
  if (isLoggedIn) {  
    return <UserGreeting />;  
  }  
  return <GuestGreeting />;  
  
  // return isLoggedIn ? <UserGreeting /> : <GuestGreeting />;  
}  
  
export default Greeting;
```

```
import Greeting from './Conditional Rendering/GreetingMain';
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <Greeting isLoggedIn={true} />  
);
```

Connect With Backend

Check the code in the classroom.

[Rendering Lists – React](#)